[CS-475] Assignment 6: Path Planning and Obstacle Avoidance

Iliana Platona

May 2026

1 Introduction

This assignment focuses on fully autonomous planar navigation in MuJoCo with both known and random obstacles. We evaluated three path planning frameworks grid-based A^*, continuous sampling-based Rapidly-exploring Random Trees (RRT), and Artificial Potential Fields (APF). In order to track the trajectories one can use an odometry-based feedback controller and a local LiDAR for collision avoidance.

2 Implementation

2.1 Task 1: Obstacle Representation

Obstacles are received on `/obstacle_boxes` as flat arrays of (c_x, c_y, h_x, h_y) tuples. Each rectangle is inflated by $r_{\text{inflate}} = r_{\text{robot}} + r_{\text{margin}} = 0.34\,\text{m}$ and the planner treats the robot as a point (object with no dimensions).

2.2 Task 2: A* Planner

The world is discretised into a 100×100-cell occupancy grid at 0.10 m resolution covering the square $[-5,\, 5]\,\text{m}$. Inflated obstacle bounds are pre-computed once per plan and used to mark cells occupied. Search uses 8-connected neighbours: cardinal moves cost r and diagonal moves cost $\sqrt{2}\,r$. The priority is $f(n) = g(n) + h(n)$ where

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} \tag{1}$$

is an admissible Euclidean heuristic. Once the goal cell is reached the path is reconstructed by tracing the parent dictionary, converted to world coordinates, shortcut-smoothed, and densified at 0.20 m spacing.

2.3 Task 3: RRT Planner

RRT builds a tree in continuous space from the start position. At each iteration a random point is sampled uniformly inside the map; with probability 0.10 the goal is sampled directly to bias exploration. The nearest node is extended toward the sample by at most 0.28 m; extensions that intersect an inflated obstacle are rejected. When a new node lies within one step of the goal and the connecting segment is free, the goal is appended and the path is reconstructed. A new candidate path is only accepted during replanning if it is at least 0.20 m shorter than the remaining current path, or if the current path has become blocked.

2.4 Task 4: Artificial Potential Field Planner

APF moves a probe point step by step toward the goal using a combination of attractive and repulsive forces. The attractive force pulls the probe toward the goal, while repulsive forces push it away from nearby obstacles and the map boundary. At each step the forces are summed, normalized, and the probe moves 0.08 m in that direction. If the probe gets stuck (no progress for 300 steps), a random nudge is injected and if the direct step is blocked by an obstacle, the force direction is rotated through eight small angles until a free step is found. If neither recovers progress, the planner falls back to A* as we will see later.

2.5 Task 5: Waypoint Tracking

The robot looks 0.75 m ahead along the planned path and steers toward that point. It computes how far away and how misaligned it is from the target, then sets forward speed proportional to distance and turning speed proportional to heading error. If the heading error is large the forward speed is reduced, and above 1.35 rad the robot stops moving forward entirely and just rotates in place until it is roughly facing the right direction.

2.6 Task 6: Local Lidar Avoidance

After the tracking controller produces a nominal command, a local safety layer inspects `/scan` across three sectors: front ($|\alpha| \leq 32$), left (30–95), and right (-95 to -30). If the minimum front range falls below 1.05 m, linear speed is scaled down proportionally. Below 0.38 m the robot stops and turns toward the side with more free space. A small angular bias of 0.55 rad/s is added away from the closer side obstacle. The layer modifies the existing `Twist` command in-place without replacing the global planner.

Parameter	Value
Grid resolution	0.10 m
Inflation radius	0.34 m (0.22 + 0.12)
RRT step size	0.28 m
RRT max iterations	4500
APF attractive gain	1.0
APF repulsive gain	0.30
APF influence dist.	0.85 m
Linear gain	1.65
Angular gain	2.45
Front slow distance	1.05 m
Front stop distance	0.38 m

Table 1: Parameter values used in all experiments.

3 Parameter Settings

4 Experiments and Results

4.1 A*, Seed 7

A^* consistently found a collision-free path from $(-4.0, -3.6)$ to $(4.0, 3.6)$. The planner produced approximately 60 waypoints at the start and the number decreased steadily as the robot moved forward, which confirms the correct replanning from the current position until the robot reached the goal. The path navigated around the UOC wall structure and all random obstacles are visible in Figure 1.

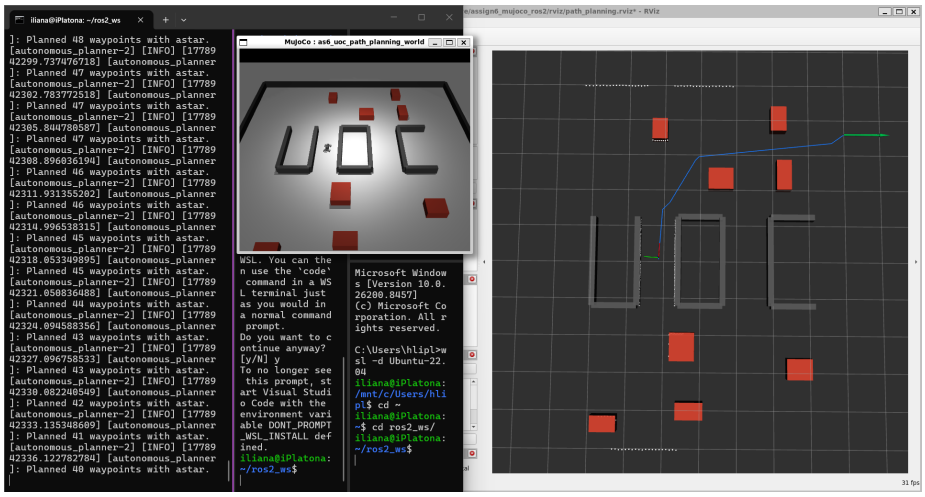

Figure 1: A^* planner, seed 7 mid-run with path in RViz and robot in MuJoCo

4.2 RRT, Seed 7

RRT also successfully navigated to the goal. The terminal consistently printed *"Keeping current RRT path"*, indicating the replanning logic correctly rejected new candidates that were not sufficiently shorter than the path already being tracked. The path is less regular than A^* due to the random sampling, but was collision-free and tracked successfully (Figure 2).

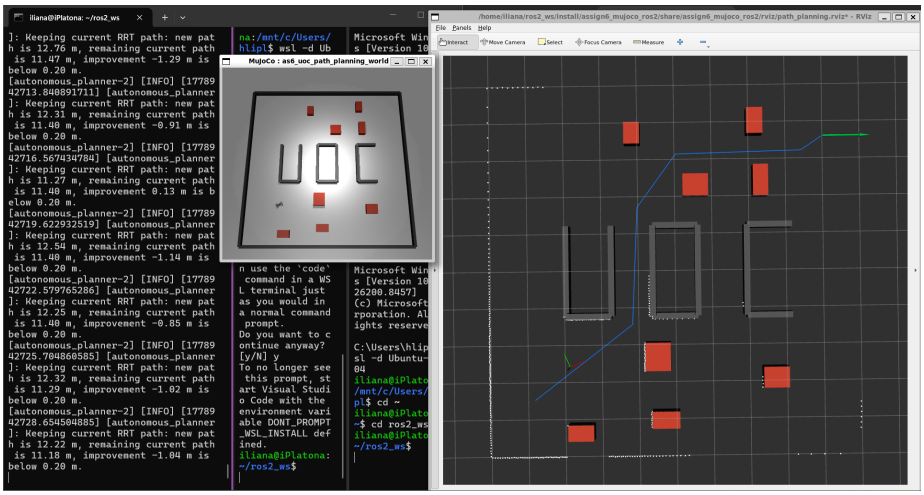

Figure 2: RRT planner, seed 7 in the start of run

4.3 APF, Seed 7

APF exhibited the expected local-minimum behaviour. The terminal reported *"APF: stuck in local minimum"* followed by *"apf failed; falling back to A*"*. Despite this, the fallback mechanism allowed the robot to continue navigating and *"Goal reached"* was printed at the end of the run (Figure 3). The local minimum occurred in the upper-right region of the map where two random obstacles and the boundary created a saddle point in the potential field.

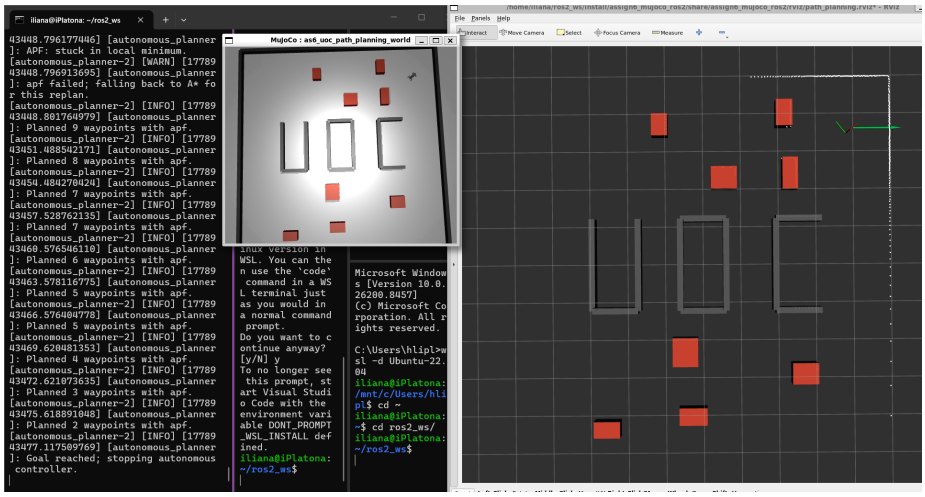

Figure 3: APF planner, seed 7 at the end of run. Terminal shows local-minimum warning and goal reached.

4.4 Second Seed (11)

Running A^* and RRT with `random_seed:=11` produced a different obstacle layout. A^* remained deterministic and found a path of comparable length. RRT produced a different tree structure due to its random sampling but still converged within the iteration budget. The main difference was that seed 11 placed obstacles closer to the direct diagonal from start to goal, forcing both planners to take a wider detour around the centre of the map.

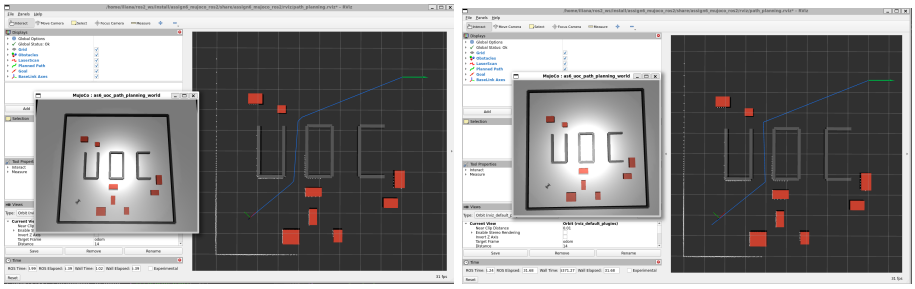

Figure 4: Seed 11 - A^* (left) and RRT (right) at the start of their runs.

4.5 10 Obstacles

With `random_obstacle_count:=10` the environment became more cluttered. A^* handled the denser layout without difficulty. RRT required more iterations in the denser space but still converged within the 4500-iteration budget. APF

struggled more as the higher obstacle density created multiple local minima and the fallback to A^* was triggered more frequently.

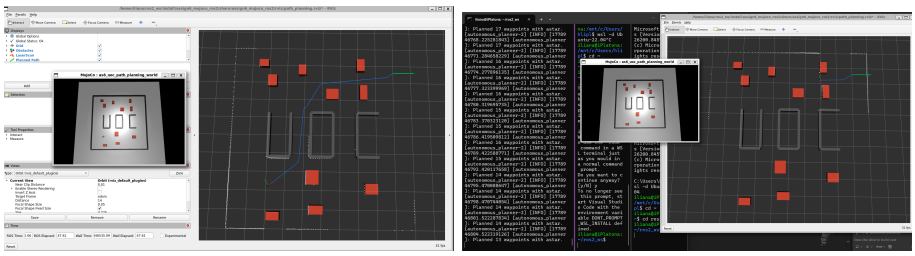

Figure 5: Ten obstacles - A^* at the start (left) and near the goal (right).

5 Planner Comparison

A^* consistently produced the shortest paths because it searches optimally on the grid. RRT paths were slightly longer and less regular. APF paths (when not falling back) were the longest because the gradient-descent trajectory curves around repulsive fields rather than taking straight shortcuts.

A^* was the most reliable planner as it is deterministic and complete on a finite grid. RRT was reliable in practice within the 4500-iteration budget but is not guaranteed to find a path. APF was the least reliable because it failed on its own in every seed-7 run and required the A^* fallback.

6 Failure Case and Debugging

The primary failure was APF getting trapped in a local minimum in the upper-right quadrant of the map near approximately (2.5, 2.5). Two random obstacles and the map boundary created a configuration where repulsive vectors partially cancelled the attractive vector, reducing the total force norm below 10^{-3}. Random perturbation injection helped temporarily but the probe returned to the same saddle region. After 300 consecutive steps without progress the planner declared failure and fell back to A^*.

7 Discussion

A^* was the best overall planner for this environment with it being deterministic, complete, and near-optimal on the 0.10 m grid. RRT is a viable alternative for higher-dimensional or continuous configuration spaces but offered no big advantage here. APF is the simplest to implement and reacts at high frequency, but its susceptibility to local minima makes it unsuitable as a standalone planner in cluttered environments.

The local LiDAR avoidance layer made the robot noticeably more robust near obstacles. Without it, the tracking controller did not react until a collision had nearly occurred. With the layer active, forward speed was reduced smoothly when the front range fell below 1.05 m, giving the angular controller enough time to steer clear.